

Shortest XOR Straight-Line Programs for 3×3 Matrix Multiplication: Benchmark Description

Greg Sidebottom

logic repository, <https://github.com/gsidebottom/logic>

Abstract—We contribute SAT instances deciding “do k XOR additions suffice?” for the output sides of rank-23 bilinear 3×3 matrix-multiplication schemes. Each boundary instance is a live mathematical question: an UNSAT answer at the right k lower-bounds the integer additive complexity of a scheme class (an optimality certificate). The exact minima are now independently known — via the transposition principle, which just produced the first sub-56 scheme (55 additions) — so these instances carry known ground-truth answers yet remain hard for every solver we tried. The family’s AND-guarded parity structure defeats current XOR/Gaussian reasoning while remaining genuinely hard for plain CDCL: no tested configuration (kissat, CaDiCaL, CryptoMiniSat plain and native-XOR, Z3 word-level, portfolios with symmetry breaking) decides the calibrated boundary cells within 10–30 minutes.

I. Problem origin

A bilinear scheme multiplying two 3×3 matrices with 23 products [8] evaluates 23 products of linear forms and combines them into the 9 output entries. Its additive complexity — binary $\pm$ additions in a straight-line program, negation free, no change of basis — has fallen rapidly: 60 [4], 59 [3], 58 [2], and the current record 56 [1]. Sun’s $56 = 13 + 13 + 30$ splits into two provably optimal input sides and an output side of 30 additions computing the 9 output forms from the 23 products.

55 additions are now achieved (13+14+28 on class i19w225c4efh), by minimizing the output side exactly via the transposition principle — the record was 56 (Sun 2026). That also yields, for every scheme, the exact output-side minimum in milliseconds, so these instances now carry known ground-truth answers (the SAT/UNSAT boundary is independently pinned) while remaining hard for every SAT/SMT solver tried; the open target is now 54.

Any $\mathbb{Z}$ -coefficient output-side program reduces mod 2 to an XOR straight-line program, so the decision problem

SLP(k): do k XOR additions suffice to compute the 9 given output forms (vectors in $\text{GF}(2)^{23}$) from the 23 inputs?

yields sound lower bounds: UNSAT at k proves the integer output side needs $\geq k+1$ additions, and UNSAT at $k = 27$ on the right cells (§III) closes entire equivalence classes for 55. These instances continue the tradition of matrix-multiplication benchmarks in past competitions [5], with a twist of independent solver interest: the parity constraints

are AND-guarded, which prevents CryptoMiniSat’s Gaussian elimination from engaging, while plain CDCL faces genuine parity hardness.

II. Encoding

SLP synthesis in the style of Fuhs–Schneider-Kamp [6], simplified by unit-vector inputs. For steps $t = 1, \dots, k$:

- Source selection. Step t selects exactly two sources among the 23 inputs and steps $1..t-1$ (sequential counter for the exactly-two constraint).
- Values. Value bits are parity-defined:

$$x_{t,i} = \text{sel}_{t,\text{base}_i} \oplus \bigoplus_{j < t} (\text{sel}_{t,\text{step}_j} \wedge x_{j,i}).$$

Unit-vector inputs make the base contribution a single literal. The AND-guarded parities are materialized as Tseitin chains (plain CNF) or native x-lines (CryptoMiniSat extension); the generator emits both.

- Outputs. Each of the 9 forms must equal some step value (selector-guarded bit equalities; weight-1 forms may match an input).
- Symmetry breaking (optional, default on): all step values nonzero; adjacent independent steps (step $t+1$ does not consume step t) must have strictly lexicographically increasing values. Sound: swapping adjacent independent steps preserves validity, so every program normalizes by bubble sort, and padding above the minimum survives as a dependent chain. Dead-step elimination is deliberately not used: with it, satisfiability is not monotone in k (odd-length padding can force a dead step), which corrupts minimum-finding by descent.

At $k = 29$ with symmetry breaking: 25,799 variables, 91,426 clauses.

III. Instances and empirical hardness

An instance is determined by (output tensor of a scheme representative, k). Tensors come from the public database and the record chain; the de Groote action makes the output form-set depend only on the pair (R, P) of $\text{GL}(3, 2)$ matrices; each of the $168 \times 168 = 28,224$ pairs is a cell (one concrete output-side instance), so every class supplies $\sim 28k$ cells — a practically unlimited supply, with k the hardness dial:

- SAT phase ($k \geq \text{minimum}$): minutes at minimum + 1; witnesses are extracted and replay-verified by the generator.
- Deep UNSAT (k well below minimum): easy.
- Boundary ($k \in \{\text{min}-1, \text{min}\}$): hard. On the calibrated seed cells no tested configuration decides the boundary within 10–30 minutes (Apple M4 Pro, one core per solver): kissat (600s and 1800s), CaDiCaL (600s), CryptoMiniSat 5 (615s, plain and native-XOR), Z3 4.16 on a word-level QF_BV formulation (600s), and a kissat+CaDiCaL+CMS portfolio with symmetry breaking (900s).

TABLE I
Calibrated seed cells (SAT witnesses verified).

cell	weight	GF(2) min
sun56 output side (record scheme)	49	$\in \{29, 30\}$
cn120 output side ($C=28$ rep)	60	$\in \{27, 28\}$

Deciding a boundary certifies a per-class optimum: e.g. an UNSAT certificate at the transposition-computed minimum proves that a class’s output side cannot be smaller, and (with the input-side exhaustions) that its total is optimal — a DRAT-checkable optimality theorem for a fixed class, now that a 55 scheme is known and the target is 54.

IV. Proposed benchmark set

Twenty instances spanning the phases: the two seed cells at $k \in \{\text{min}-1, \text{min}, \text{min}+1\}$; the identity cells of the two other known 56-addition classes (i12w219c23ci, i19w225c4efh) at their boundaries; and four fat-sides window cells of the record class at $k = 27$; each with and without symmetry breaking. All are emitted by the C-side lower-bound generator (cxl.py):

```
python3 matmul/cxl.py --bits SCHEME.bits \
--k K --dump out.cnf
python3 matmul/cxl.py --bits SCHEME.bits \
--k K --dump out.xnf # native XOR
```

(-no-sb disables symmetry breaking; all scheme bits files are committed in the repository.)

V. Availability

Generator, scheme inputs, verification tooling, and research notes: <https://github.com/gsidebottom/logic>. Submitted instances may be used freely under the competition’s standard terms.

Appendix

The formulation of Section II is also realized, constraint for constraint, as a formula over box constraints for the repository’s matrix-method solver (lib/slp.jq; the web app’s boxes backend, doc/box_backend_design.md). A box is a compiled table constraint called by name in

the formula language, name(args), propagated to arc consistency — or, when the table and its negation are tiny, as the equivalent clauses. Five boxes suffice:

- cnt2(u1;u2;s;v1;v2) — one step of the sequential counter, $v_1 = u_1 \vee s$, $v_2 = u_2 \vee (u_1 \wedge s)$, $\neg(u_2 \wedge s)$; a chain from the constants (0,0) to (1,1) over the selectors $s_{t,0}, \dots, s_{t,n+t-1}$ is “exactly two sources”.
- gx(p;s;x;q) — $q = p \oplus (s \wedge x)$; a chain from $s_{t,i}$ through the earlier steps is the AND-guarded parity $x_{t,i} = s_{t,i} \oplus \bigoplus_{u < t} (s_{t,n+u} \wedge x_{u,i})$ (step 0’s value bits are its input selectors).
- imp(a;b) — $a \Rightarrow b$; imp(o_f_t;x_t_i) or imp(o_f_t;x_t_i') per bit is the selector-guarded equality of form f with step t ’s value.
- orb(p;o1;...;o7;q) — $q = p \vee o_1 \vee \dots \vee o_7$; a chain ending in the constant 1 is “at least one” (some step matches the form; every step value nonzero).
- lexstep(e;g;a;b;e2;g2) — $e_2 = e \wedge (a = b)$, $g_2 = g \vee (e \wedge \neg a \wedge b)$; a chain over the bits from $n-1$ down, from (1,0), then imp(s_{t+1}_{n+t});g, is the lexicographic symmetry breaking of adjacent independent steps.

Forms of weight 1 are dropped (an input computes them). An instance is $\{n, \text{forms}\}$; slp(inst; k) generates SLP(k) with symmetry breaking, slp(inst; k; false) without. The seed cells’ output forms are embedded as data (sun56_cell, cn120_cell, i19_cell, i12_cell, read from the .bits files as cxlb.py reads them) together with Strassen’s output side over $\mathbb{F}_2$ (strassen_out: 4 forms over 7 products); slp_window(cell; [indices]) restricts a cell to the chosen forms over the inputs they use. The same formula serves CaDiCaL: the web app expands the calls into their definitions and Tseitin-encodes them, so both engines see the identical constraint set (for the main instances the generator’s own CNF remains the reference).

a) Reading a program off a witness: tools/slp_program.py runs an instance on either engine, reads each step’s two sources from the $s_{t,j}$ of the witness, replays the program over $\mathbb{F}_2^n$ and checks that every form is the value of some step (the replay verification of Section III); Fig. 1 shows three runs. Strassen’s output side needs exactly 8 additions over $\mathbb{F}_2$.

b) Tractable windows: tools/slp_bench.py descends k with CaDiCaL to each instance’s minimum and times both engines at min-1, min, min+1 under a 60-second cap; Table II is the run of 2026-09-15, with the engine’s minimal hitting-set explanations, decision heap and learned-clause minimisation (Apple M4 Pro, one core per solver, times as the web app reports them). Windows of three forms with $n \leq 12$ and minimum ≤ 10 are decidable by both engines within the budget, which places them well below the seed cells of Section III on the hardness dial. The box engine refutes the min-1 rows in 0.4–1.3s where CaDiCaL needs 0.2–0.35s; on the satisfiable rows it is 3–28 $\times$ faster than CaDiCaL on six (i19[4,6,7] at $k = 8$ in 0.01s against 0.28s) and 7–14 $\times$ slower on

```

$ tools/slp_program.py strassen_out 8
boxes backend: SLP(8) on strassen_out (n = 7, 4 forms of weight >= 2, 576 boxes): SAT in 0.18s
y1 = x1 + x2 = 1100000
y2 = x3 + y1 = 1110000
y3 = x2 + x4 = 0101000
y4 = x3 + x5 = 0010100
y5 = x6 + y2 = 1110010
y6 = x7 + y1 = 1100001
y7 = x5 + y6 = 1100101
y8 = y3 + y7 = 1001101
form 1001101: y8
form 0010100: y4
form 0101000: y3
form 1110010: y5
replay: every form computed

$ tools/slp_program.py strassen_out 7
boxes backend: SLP(7) on strassen_out (n = 7, 4 forms of weight >= 2, 472 boxes): UNSAT in 0.47s

$ tools/slp_program.py 'slp_window(sun56_cell; [0, 5, 7])' 8 --cadical
CaDiCaL: SLP(8) on slp_window(sun56_cell; [0, 5, 7]) (n = 11, 3 forms of weight >= 2, 794 boxes): SAT in 0.26s
y1 = x5 + x6 = 00001100000
y2 = x9 + y1 = 00001100100
y3 = x7 + x10 = 00000010010
y4 = x3 + y3 = 00100010010
y5 = x1 + x11 = 10000000001
y6 = x8 + y5 = 10000001001
y7 = x2 + y6 = 11000001001
y8 = x4 + y7 = 11010001001
form 00001100100: y2
form 11010001001: y8
form 00100010010: y4
replay: every form computed

```

Fig. 1. Decoded, replay-verified programs: Strassen’s output side at $k = 8$ (SAT) and $k = 7$ (UNSAT) on the boxes backend, and a three-form window of the record cell on CaDiCaL.

TABLE II
Tractable windows (cell[form indices]); each cell of the table is $k = \min -1 / \min / \min +1$.

instance	n	weights	k	boxes backend	CaDiCaL
strassen_out	7	4,2,2,4	7 / 8 / 9	UNSAT 0.4 s / SAT 0.1 s / SAT 0.01 s	UNSAT 0.2 s / SAT 0.01 s / SAT 0.04 s
sun56[0,3,7]	9	3,3,3	5 / 6 / 7	UNSAT 0.01 s / SAT 0.00 s / SAT 0.03 s	UNSAT 0.02 s / SAT 0.01 s / SAT 0.02 s
sun56[0,5,7]	11	3,5,3	7 / 8 / 9	UNSAT 1.2 s / SAT 0.07 s / SAT 0.03 s	UNSAT 0.3 s / SAT 0.2 s / SAT 0.1 s
i12[0,1,3]	11	3,5,3	7 / 8 / 9	UNSAT 1.3 s / SAT 0.02 s / SAT 1.3 s	UNSAT 0.3 s / SAT 0.4 s / SAT 0.2 s
i19[4,6,7]	11	3,5,3	7 / 8 / 9	UNSAT 1.0 s / SAT 0.01 s / SAT 0.01 s	UNSAT 0.3 s / SAT 0.3 s / SAT 0.2 s
cn120[0,6,8]	9	5,5,3	7 / 8 / 9	UNSAT 1.2 s / SAT 0.5 s / SAT 0.2 s	UNSAT 0.2 s / SAT 0.5 s / SAT 0.2 s
i12[0,3,4]	12	3,3,7	9 / 10 / 11	> 60 s / SAT 4.9 s / SAT 0.8 s	UNSAT 10.1 s / SAT 0.4 s / SAT 0.4 s
sun56[1,2,4]	14	7,7,7	13 / 14 / 15	> 60 s / > 60 s / SAT 23.0 s	> 60 s / SAT 15.7 s / SAT 58.3 s

three (i12[0,3,4] at $k = 10$ in 4.9s against 0.43s), and it solves the window of three weight-7 forms at $k = 15$ in 23s where CaDiCaL takes 58s; the $n = 12$ boundary (CaDiCaL 10s) and that window’s $k = 13$ and 14 rows are beyond it. The AND-guarded parities defeat table propagation exactly as they defeat Gaussian elimination—arc consistency on the gx chains forces nothing that unit propagation on the same clauses does not—and with the engine’s earlier trail-order explanations the satisfiable rows were 30–150× slower than CaDiCaL. What the tables contribute is the explanation: a minimal hitting-set reason of two or three literals where the trail-order one was longer, and correspondingly stronger learned clauses.

Acknowledgments

Built in an extended interactive collaboration with Claude (Anthropic; the Fable 5 and Opus 4.8 models), which implemented the generators and verifiers under the

author’s direction and review; all instances and certificates are mechanically checkable.

References

- [1] Y. Sun. An Exact 56-Addition, Rank-23 Scheme for General 3×3 Matrix Multiplication. arXiv:2604.27645, 2026.
- [2] A. I. Perminov. A 58-Addition, Rank-23 Scheme for General 3×3 Matrix Multiplication. arXiv:2512.21980, 2025.
- [3] E. Mårtensson, P. Stankovski Wagner, J. Stapleton. A Rank 23 Algorithm for Multiplying 3×3 Matrices with an Arithmetic Complexity of 59. arXiv:2601.05272, 2025.
- [4] J. Stapleton. A 60-Addition, Rank-23 Scheme for Exact 3×3 Matrix Multiplication. arXiv:2508.03857, 2025.
- [5] M. J. H. Heule, M. Kauers, J. Seidl. Local search for fast matrix multiplication. SAT 2019; J. Symbolic Computation 104, 2021.
- [6] C. Fuhs, P. Schneider-Kamp. Synthesizing shortest linear straight-line programs over GF(2). SAT 2010.
- [7] J. Boyar, R. Peralta. A new combinational logic minimization technique with applications to cryptology. SEA 2010.
- [8] J. Laderman. A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications. Bull. AMS 82(1), 1976.